



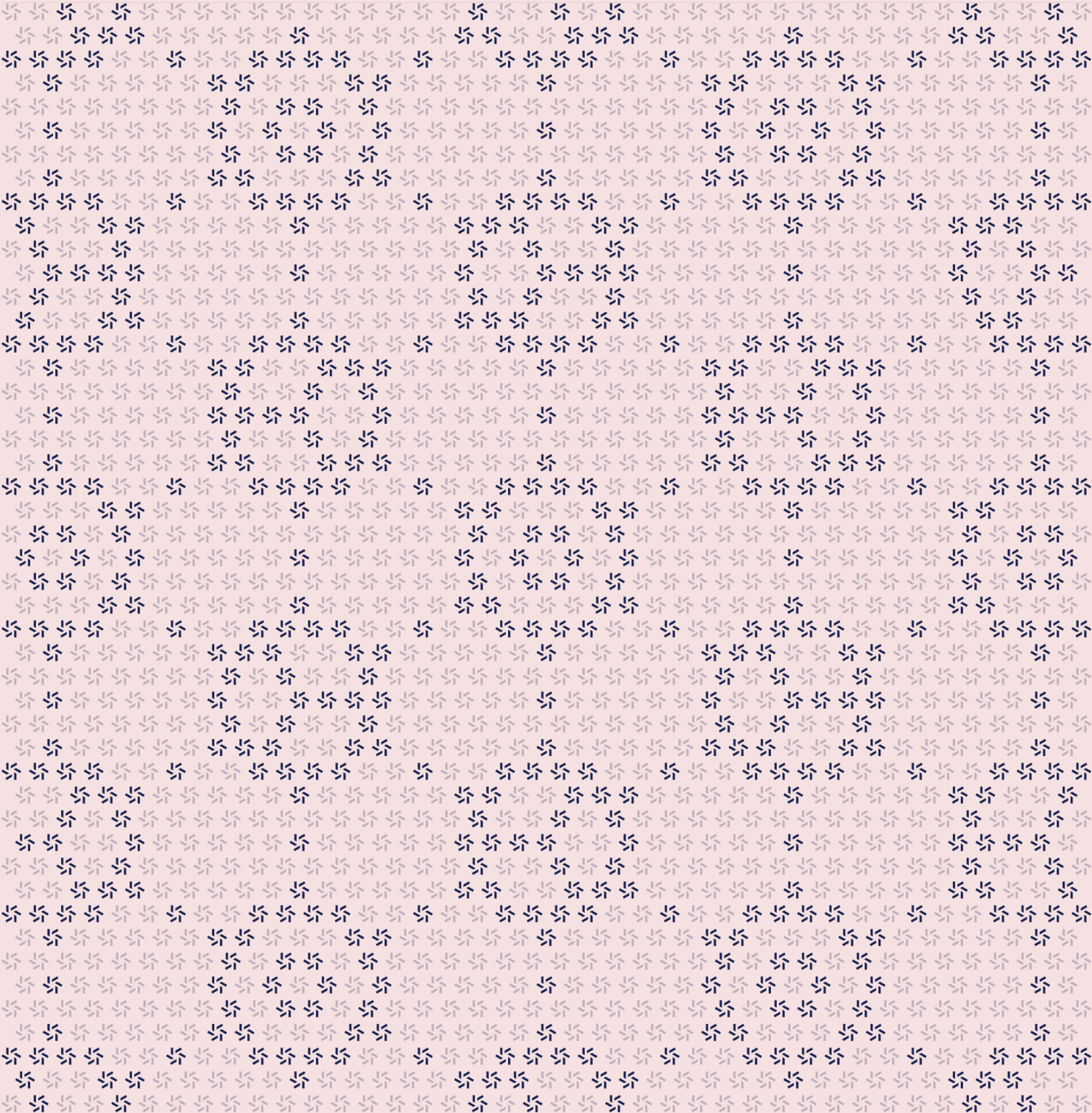
Prepared for
Julien Genestoux
Peter Watts
Jason Maier
Uneven Labs, Inc.

Prepared by
Aaron Esau
Sihoon Lee
Daniel Lu
Zellic

November 21, 2025

Relay Settlement Protocol

Smart Contract Security Assessment



Contents

About Zellic	4
1. Overview	4
1.1. Executive Summary	5
1.2. Goals of the Assessment	5
1.3. Non-goals and Limitations	5
1.4. Results	5
2. Introduction	6
2.1. About Relay Settlement Protocol	7
2.2. Methodology	7
2.3. Scope	9
2.4. Project Overview	11
2.5. Project Timeline	12
3. Detailed Findings	12
3.1. Domain separator missing in Solana relay depository	13
3.2. RelayHub allows for arbitrary token transfers	15
3.3. Critical variables are not signed in signWithdrawPayloadHash	17
3.4. Denial of service by parking nonces	19
3.5. Denial of service by indefinitely pushing signature time into the future	20
3.6. Missing nonce lifetime validation for Hyperliquid withdrawals	21
3.7. ECDSA signature replay	22
3.8. Withdrawer may create multiple signatures for one withdrawal	23

3.9.	Insufficient test coverage	25
3.10.	Block producers may reassign deposit credit	27
3.11.	Missing commitments to chain and contract might allow replay	29
4.	Discussion	30
4.1.	Consider adding an address (0) check for the payload builder	31
4.2.	TON escrow coding practices	31
4.3.	TON escrow execution has loose ordering	33
4.4.	TON escrow implementation has fewer features	33
5.	Threat Model	33
5.1.	Module: RelayAllocator.sol	34
5.2.	Module: RelayDepository.sol	48
5.3.	Module: RelayMultisigSigner.sol	53
6.	Assessment Results	57
6.1.	Disclaimer	58

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io and follow [@zellic_io](#) on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io.



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Uneven Labs, Inc. from October 16th to November 12th, 2025. During this engagement, Zellic reviewed Relay Settlement Protocol's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Are funds safe when deposited in the Depository contract?
 - Are withdrawals safe on EVM, Solana, and Bitcoin chains?
 - Are access controls properly implemented to prevent unauthorized operations?
-

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

During our assessment on the scoped Relay Settlement Protocol files, we discovered 11 findings. One critical issue was found. Two were of high impact, five were of medium impact, two were of low impact, and the remaining finding was informational in nature.

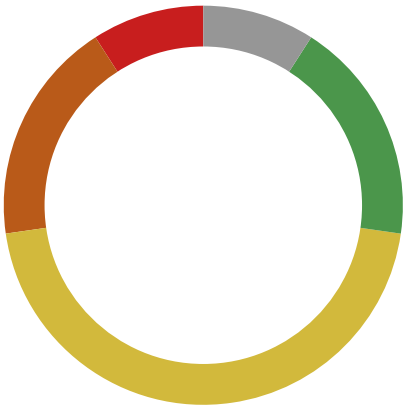
Additionally, Zellic recorded its notes and observations from the assessment for the benefit of Uneven Labs, Inc. in the Discussion section ([4. 7](#)).

While we are confident in the effectiveness of the current fixes, the volume of the findings, combined with the constraints of a time-boxed engagement, indicates that additional review would be beneficial to help identify and address any remaining issues in the original codebase as well as any

new vulnerabilities introduced by recent changes. We strongly recommend a follow-up assessment prior to deployment. We also recommend adopting a security-focused development workflow, including (but not limited to) augmenting the repository with comprehensive end-to-end tests that achieve 100% branch coverage using any common, maintainable testing framework, thoroughly documenting all function requirements, and training developers to have a security mindset while writing code. More details about our recommendation can be found in Section [6.7](#).

Breakdown of Finding Impacts

Impact Level	Count
Critical	1
High	2
Medium	5
Low	2
Informational	1



2. Introduction

2.1. About Relay Settlement Protocol

Uneven Labs, Inc. contributed the following description of Relay Settlement Protocol:

Relay Settlement Protocol is a non-custodial intent protocol. Users are making deposits to have their intents fulfilled by the Relay Solver. The Relay Solver withdraws user-deposited funds once their intents have been filled.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the files.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood.

We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped files itself. These observations — found in the Discussion (4. 7) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

2.3. Scope

The engagement involved a review of the following targets:

Relay Settlement Protocol Files

Types	Rust, Solidity, FunC, Sui Move
Platforms	SVM-compatible, EVM-compatible, TON, Sui
Target	Relay Depository
Repository	https://github.com/relayprotocol/relay-depository ↗
Version	aec671d3e3e7b72135cf2d570eb6cc59e757908b
Programs	ethereum-vm/src/**/*.sol solana-vm/programs/relay-depository/src/**/*.rs solana-vm/programs/relay-forwarder/src/**/*.rs sui-vm/relay-escrow/sources/**/*.move ton-vm/contracts/**/*.fc

Target	Allocator and PayloadBuilders
Repository	https://github.com/relayprotocol/decentralized-protocol ↗
Version	15ba05ba0ae1e65e1d72c4ddeb18312568802f45
Programs	ChainSignatures.sol RelayAllocator.sol Utils.sol PayloadBuilders/BitcoinPayloadBuilder.sol PayloadBuilders/EVMPayloadBuilder.sol PayloadBuilders/SolanaPayloadBuilder.sol PayloadBuilders/SuiPayloadBuilder.sol
Target	PayloadBuilder (Hyperliquid)
Repository	https://github.com/relayprotocol/decentralized-protocol ↗
Version	15ba05ba0ae1e65e1d72c4ddeb18312568802f45
Programs	PayloadBuilders/HyperliquidPayloadBuilder.sol
Target	RelayMultisigSigner
Repository	https://github.com/relayprotocol/decentralized-protocol ↗
Version	15ba05ba0ae1e65e1d72c4ddeb18312568802f45
Programs	RelayMultisigSigner.sol

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of four person-weeks. The assessment was conducted by three consultants over the course of four calendar weeks.

Contact Information

The following project manager was associated with the engagement:

Jacob Goreski
✧ Engagement Manager
jacob@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Aaron Esau
✧ Engineer
aaron@zellic.io ↗

Sihoon Lee
✧ Engineer
sihoon@zellic.io ↗

Daniel Lu
✧ Engineer
daniel@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

October 15, 2025	Kick-off call
October 16, 2025	Start of EVM and Solana review period
October 31, 2025	End of EVM and Solana review period
November 11, 2025	Start of TON and Sui review period
November 12, 2025	End of TON and Sui review period

3. Detailed Findings

3.1. Domain separator missing in Solana relay depository

Target	Solana relay-depository		
Category	Coding Mistakes	Severity	Critical
Likelihood	Medium	Impact	Critical

Description

On the Solana relay depository, the signature verification does not include a domain separator (e.g., EIP-712 style) when signing messages.

```

/// Structure representing a transfer request signed by the allocator
#[derive(AnchorSerialize, AnchorDeserialize, Copy, Clone, PartialEq, Debug)]
pub struct TransferRequest {
    /// The recipient of the transfer
    pub recipient: Pubkey,
    /// The token mint (None for native SOL, Some(mint) for SPL tokens)
    pub token: Option<Pubkey>,
    /// The amount to transfer
    pub amount: u64,
    /// A unique nonce
    pub nonce: u64,
    /// The expiration timestamp for the request
    pub expiration: i64,
}

impl TransferRequest {
    /// Computes a hash of the serialized request for signature verification
    /// and used request tracking
    pub fn get_hash(&self) -> Hash {
        hash(&self.try_to_vec().unwrap())
    }
}

// [...]
fn validate_ed25519_signature_instruction(
    signature_ix: &Instruction,
    expected_signer: &Pubkey,
    expected_request: &TransferRequest,
) -> Result<()> {
    // [...]
    // Verify message hash matches request hash

```

```
let expected_hash = expected_request.get_hash().to_bytes();
if data_msg != expected_hash {
    return Err(CustomError::MessageMismatch.into());
}

Ok(())
```

This means that signatures intended for any SVM-compatible chains, including Solana, Eclipse, or Soon, can be replayed on all of them.

Similarly, if the depository were to be forked and deployed on the same chain multiple times, then signatures could be reused between the deployments, because the program ID is not included in the signed message (in the form of a domain separator).

Impact

An attacker can execute transfers multiple times across all supported SVM-compatible chains.

Recommendations

Include a domain separator in the signed message.

Remediation

This issue has been acknowledged by Uneven Labs, Inc., and fixes were implemented in the following PRs:

- [PR #32](#) ↗
- [PR #33](#) ↗
- [PR #203](#) ↗

They intend to merge these changes into the production branch.

3.2. RelayHub allows for arbitrary token transfers

Target	RelayHub		
Category	Coding Mistakes	Severity	Critical
Likelihood	High	Impact	Informational

Description

Note: This critical issue falls outside the defined scope of the assessment, so it has been assigned an Impact rating of Informational. However, because it was discovered during the assessment period and has security implications for the protocol, we are documenting it here for completeness and to ensure it is properly remediated.

RelayHub's `transferFrom` has special logic to allow the `ERC20View` contract to perform arbitrary transfers, skipping the allowance checks:

```
function transferFrom(
    address sender,
    address receiver,
    uint256 id,
    uint256 amount
) public returns (bool result) {
    if (
        msg.sender != sender &&
        !isOperator(sender, msg.sender) &&
        erc20Views[id] != msg.sender
    ) {
        uint256 allowed = allowance[sender][msg.sender][id];
        if (allowed != type(uint256).max)
            allowance[sender][msg.sender][id] = allowed - amount;
    }

    balanceOf[sender][id] -= amount;

    balanceOf[receiver][id] += amount;

    emit Transfer(msg.sender, sender, receiver, id, amount);
    _emitERC20ViewEvent(id, sender, receiver, amount);
}
```

```
        return true;
    }
```

This is problematic because the ERC20View contract also has a `transferFrom` function, which simply passes its arguments to the RelayHub's `transferFrom` function:

```
/// @notice Transfers tokens from one address to another
/// @param from The sender address
/// @param to The recipient address
/// @param value The amount to transfer
/// @return True if successful
function transferFrom(
    address from,
    address to,
    uint256 value
) public returns (bool) {
    // Execute the transfer on the Hub (Hub will handle allowance checks and
    // updates)
    bool success = hub.transferFrom(from, to, tokenId, value);
    if (success) {
        emit Transfer(from, to, value);
    }
    return success;
}
```

If called through the ERC20View contract, from the RelayHub's perspective, the `msg.sender` will be the ERC20View contract, so allowance checks will be skipped.

Impact

Anyone can transfer any amount of anyone's RelayHub tokens, at any time.

Recommendations

Add a separate function to handle ERC20View's trusted transfers, instead of reusing `transferFrom`. And ensure that RelayHub's `transferFrom` never skips allowance checks.

Remediation

This issue has been acknowledged by Uneven Labs, Inc., and a fix was implemented in [PR #171](#). They intend to merge these changes into the production branch.

3.3. Critical variables are not signed in signWithdrawPayloadHash

Target	RelayAllocator		
Category	Coding Mistakes	Severity	Critical
Likelihood	High	Impact	High

Description

The `signWithdrawPayloadHash` function allows anyone with a valid signature to request NEAR signatures. It calls `verifyWithdrawal`, which calls `signatureMatchesReceiver` to ensure the signature matches the receiver and to mark the nonce as used.

However, the caller can control two arguments, which are not in the payload that is checked to have been signed by the receiver:

```
function signWithdrawPayloadHash(
    SubmitWithdrawRequest calldata params,
    bytes memory signature,
    GasSettings memory gasSettings,
    uint32 hashIndex
) public {
    // [...]
    // verify the withdrawal can be achieved
    verifyWithdrawal(params, payloadBuilder, signature);

    // get the hash to sign
    bytes32 hashToSign = payloadBuilder.hashToSign(
        params.chainId,
        params.depositary,
        payload,
        hashIndex
    );

    _signUsingChainSignatures(
        withdrawRequestHash,
        hashToSign,
        payloadBuilder,
        gasSettings
    );
}
```

This is problematic, because once a signature is obtained for a given receiver and nonce, the caller can call `signWithdrawPayloadHash` to park the nonce with invalid gas settings or hash index.

Even if the user uses `submitAndSignWithdrawRequest`, which calls the `_submitWithdrawRequest` and `signWithdrawPayloadHash`, it is possible to front-run the transaction and use the same params and signature.

Impact

An attacker can cause a denial of service by consuming the nonce with invalid gas settings such that the NEAR signature is never created.

More importantly, the ability to control `hashIndex` means that the attacker can cause the transaction to spend unintended inputs, which could lead to loss of user funds.

Recommendations

The `gasSettings` and `hashIndex` should be included in the signed payload (`SubmitWithdrawRequest`) to ensure that the caller cannot manipulate them after obtaining a valid signature.

Remediation

This issue has been acknowledged by Uneven Labs, Inc..

They also added the following note regarding this finding:

This affects a flow that we will actually never use because the Hub contract will never be deployed on Aurora and we will keep relying on the permissioned role in the short term. Long term, we will decouple the signing logic from Near's chain signature and we will deploy a new version of this contract for which we will take this into account. We will also document explicitly that this flow won't be used in production.

3.4. Denial of service by parking nonces

Target	RelayAllocator		
Category	Business Logic	Severity	High
Likelihood	High	Impact	High

Description

The RelayAllocator contract has a usedNonces mapping that is shared between all depositories. If a depository requires a delay for a withdrawal request, then there is a period between when the victim creates the withdrawal request and when it can be signed.

In that time, an attacker can create a request — copying the nonce of the victim's request — on a depository that does not require a delay (or has a delay shorter than the victim's depository) and mark the nonce as used in usedNonces, thereby blocking the victim's request.

Impact

An attacker can prevent a victim from being able to create withdrawal requests, causing a denial of service.

Recommendations

Our recommendation to mitigate this is to use an incremental nonce keyed by the sender.

Remediation

This issue has been acknowledged by Uneven Labs, Inc., and a fix was implemented in [PR #173](#). They intend to merge these changes into the production branch.

3.5. Denial of service by indefinitely pushing signature time into the future

Target	RelayAllocator		
Category	Coding Mistakes	Severity	High
Likelihood	High	Impact	Medium

Description

The `submitWithdrawRequest` allows anyone to call it any number of times with the same arguments. The state changes it makes includes updating the `payloadTimestamps` mapping to configure the time when the withdrawal can be signed.

However, since it is callable many times, an attacker can repeatedly call it on a user's request indefinitely, pushing the withdrawal off into the future.

Impact

An attacker can prevent a withdrawal from being signed for a long time, causing a denial of service.

Recommendations

Add a check to ensure that the `submitWithdrawRequest` can only be called once per unique withdrawal request.

Remediation

This issue has been acknowledged by Uneven Labs, Inc., and a fix was implemented in [PR #174](#). They intend to merge these changes into the production branch.

3.6. Missing nonce lifetime validation for Hyperliquid withdrawals

Target	RelayAllocator		
Category	Business Logic	Severity	High
Likelihood	Low	Impact	Medium

Description

According to the [Hyperliquid documentation](#) ⁷,

Nonces must be within $(T - 2 \text{ days}, T + 1 \text{ day})$, where T is the unix millisecond timestamp on the block of the transaction.

At the time of the audit, the RelayAllocator contract does not validate that the submitted nonce values fall within this valid time window. Users can submit withdrawal requests with nonces outside the acceptable range, which will be processed by the contract but rejected by the Hyperliquid chain.

Once a nonce is marked as used in the contract, it cannot be reused even if the transaction failed on the destination chain due to an invalid nonce value. This results in the withdrawal request becoming unprocessable, effectively locking the user's funds in the allocator contract.

Impact

Incorrectly submitted nonces result in locked funds in the allocator contract. While this would typically be user error, it can be easily prevented with a simple validation check.

Recommendations

Add validation to ensure nonce values fall within $[T - 2 \text{ days}, T + 1 \text{ day})$ before processing withdrawal requests. This simple check will effectively prevent users from accidentally locking their funds due to invalid nonce submission.

Add an error-handling mechanism in case an error occurs off-chain.

Remediation

This issue has been acknowledged by Uneven Labs, Inc., and a fix was implemented in [PR #186](#) ⁸. They intend to merge these changes into the production branch.

3.7. ECDSA signature replay

Target	RelayMultisigSigner		
Category	Coding Mistakes	Severity	High
Likelihood	Low	Impact	Medium

Description

Once an owner approves of a hash in RelayMultisigSigner, anyone can call `sign` to sign the same message as many times as they choose.

While ECDSA will always have a deterministic signature for the same `(message, signer)`, ECDSA has a nonce `k`, so it will produce a new signature each time. Though most chains implement a nonce at the transaction layer (i.e., inside the signed data), using this contract with a specific implementation of a signature verifier will cause it to be exploitable. If the verifier tracks spent messages by storing the signature (one way or another) and preventing stored signatures from being reused, then anyone will be able to replay ECDSA signatures as many times as they choose.

Also, since the `sign` function does not receive wNEAR from the caller, anyone can spend the contract's balance by calling the function repeatedly with an approved hash + curve combo. This is unlikely to be problematic because of gas prices and the fact that the signature costs 1 yoctoNEAR to store.

Impact

The contract is only safe for specific methods of implementing a verifier. It would be trivial to unintentionally choose a verifier for which this contract is exploitable (which allows replaying ECDSA signatures).

Recommendations

Ensure users can only request one signature per approved hash + curve combination, while still allowing for retries if the signature creation on NEAR fails for any reason.

Remediation

This issue has been acknowledged by Uneven Labs, Inc., and a fix was implemented in [PR #176](#). They intend to merge these changes into the production branch.

3.8. Withdrawer may create multiple signatures for one withdrawal

Target	RelayAllocator		
Category	Business Logic	Severity	Low
Likelihood	Low	Impact	Low

Description

Since there is the opportunity for transactions to land between the time when the `_signUsingChainSignatures` function is called and when `signWithdrawCallback` (which sets the `signedPayloads`) is called, the `_signUsingChainSignatures` function could be called multiple times, resulting in multiple signatures being created — thus making that check ineffective.

```
function _signUsingChainSignatures(
    bytes32 withdrawRequestHash,
    bytes32 hashToSign,
    IPayloadBuilder payloadBuilder,
    GasSettings memory gasSettings
) internal {
    if (signedPayloads[withdrawRequestHash][hashToSign].length > 0) {
        revert PayloadAlreadySigned(withdrawRequestHash);
    }
}
```

Impact

The impact is limited because `verifyWithdrawal` would mark the nonce as used — that is, unless the caller is the withdrawer role.

However, the withdrawer role, which is a trusted role, would be able to create multiple signatures for a single withdrawal request.

Recommendations

The check provides no security benefit regardless of its intention, and it costs additional gas to deploy or execute.

We recommend reconsidering the logic to ensure multiple signatures cannot be created for a single withdrawal request. Alternatively, remove the check entirely since it is redundant with the nonce check — but accept that the withdrawer role must be trusted.

Remediation

This issue has been acknowledged by Uneven Labs, Inc., and a fix was implemented in [PR #198](#). They intend to merge these changes into the production branch.

3.9. Insufficient test coverage

Target	Project-wide		
Category	Protocol Risks	Severity	Medium
Likelihood	N/A	Impact	Medium

Description

When building a complex contract with multiple moving parts (state changes) and dependencies, comprehensive testing is essential. This includes testing for both positive and negative scenarios on every function that touches the contract's state, integration tests to help with identifying bugs associated with more complex state changes, and tests for adversarial account scenarios.

Positive tests should verify that each function's side effect is as expected, while negative tests should cover every revert, preferably in every logical branch.

While the test suite covers most of the positive cases (though not all), there is little coverage for negative scenarios. Unit test cases that are missing include (but are not necessarily limited to) those documented in section [5.7](#).

The test coverage for this project should be expanded to include all contracts, and all functions — not just surface-level functions — for all positive and negative cases. It is important to test the invariants required for ensuring security.

Additionally, we believe that Relay Settlement Protocol can benefit from a complete integration test suite to ensure the system operates properly from end to end for supported chains.

Impact

Good test coverage has multiple effects:

- It finds bugs and design flaws early (preaudit or prerelease).
- It gives insight into areas for optimization (e.g., gas cost).
- It displays code maturity.
- It bolsters customer trust in the product.
- It improves understanding of how the code functions, integrates, and operates — for developers and auditors alike.
- It increases development velocity long-term.

The last point may seem contradictory given the time investment to create and maintain tests. To expand upon it, tests help developers trust their own changes. It is difficult to know if a code refactor — or even just a small one-line fix — breaks other code if there are no tests. This is

especially true for new developers or those returning to the code after a prolonged absence.

Tests have your back here. They are an excellent indicator that the existing functionality was most likely not broken by a change to the code. Without a comprehensive test suite, the above benefits are lost. This increases the likelihood of bugs and vulnerabilities going unnoticed until after deployment, which can be costly and damaging to the project's reputation.

Recommendations

We recommend building a rigorous test suite that includes all contracts to ensure that the system operates securely and as intended.

Remediation

This issue has been acknowledged by Uneven Labs, Inc..

3.10. Block producers may reassign deposit credit

Target	RelayDepository		
Category	Business Logic	Severity	High
Likelihood	Low	Impact	Low

Description

The RelayDepository contract includes an entry point for depositing ERC-20 tokens by allowance. As input, it takes a token address and a depositor address.

This action takes custody of the sender's entire allowance, crediting the deposit to the depositor in the emitted event.

```

/// @notice Deposit erc20 tokens and emit an `RelayErc20Deposit` event
/// @param depositor The address of the depositor - set to `address(0)` to
    credit `msg.sender`
/// @param token The erc20 token to deposit
/// @param id The identifier associated with the deposit
/// @dev Uses the full allowance granted to this contract and calls
    depositErc20
function depositErc20(
    address depositor,
    address token,
    bytes32 id
) external {
    uint256 amount = ERC20(token).allowance(msg.sender, address(this));

    depositErc20(depositor, token, amount, id);
}

```

In some situations, the fact that it uses the entire allowance may be manipulated by block producers. Suppose the original token custodian performs the following transactions to deposit on behalf of users Alice and Carol.

1. Approve 10 tokens to the RelayDepository contract.
2. Deposit the entire approval on behalf of Carol.
3. Approve 20 tokens to the RelayDepository contract.
4. Deposit the entire approval on behalf of Alice.

If Carol can control how the transactions are assembled, she may be able to reorder the transactions to swap steps 2 and 4, assigning the greater allocation to herself.

Impact

The block producer can manipulate how deposit credits are assigned, against the intentions of the original token custodian. This may include increasing or decreasing deposits from the intended amounts as well as reallocating deposits among multiple users.

In the situation above, the custodian can mitigate this issue by performing the approve/deposit sequences atomically or by using the `depositErc20` entry point that accepts an explicit amount.

Recommendations

We recommend either removing this functionality altogether or clearly documenting the risks of using this deposit entry point.

Remediation

This issue has been acknowledged by Uneven Labs, Inc..

3.11. Missing commitments to chain and contract might allow replay

Target	relay_escrow.fc		
Category	Business Logic	Severity	High
Likelihood	Low	Impact	Medium

Description

The data meant for execution is signed by the allocator. In particular, the hash signed is constructed as follows.

```
cell signing_message = begin_cell()
    .store_uint(msg_nonce, 64)
    .store_uint(expiry, 32)
    .store_uint(currency_type, 8)
    .store_slice(to_addr)
    .store_slice(jetton_addr)
    .store_slice(currency_addr)
    .store_coins(amount)
    .store_coins(forward_amount)
    .store_coins(gas_amount)
    .end_cell();
```

Missing from this hash is any reference to the chain or the specific instance of the escrow contract.

Impact

If multiple instances of the escrow contract are deployed to reference the same allocator, signatures may be reused to provoke unexpected behavior on other instances. This can result in the loss of user funds.

Recommendations

We recommend including at least the chain and contract instance into the hash.

Additionally, it would be better if the message contained a reference to the domain (such as a name and version of the protocol).

Remediation

This issue has been acknowledged by Uneven Labs, Inc., and a fix was implemented in [PR #35](#). They intend to merge these changes into the production branch.

4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

4.1. Consider adding an address(0) check for the payload builder

We recommend adding the following check:

```
function signWithdrawPayloadHash(
    SubmitWithdrawRequest calldata params,
    bytes memory signature,
    GasSettings memory gasSettings,
    uint32 hashIndex
) public {
    // [...]
    address builder = payloadBuilders[params.chainId][params.depositary];
    if (builder == address(0)) {
        revert NoPayloadBuilder(params.chainId, params.depositary);
    }
    IPayloadBuilder payloadBuilder = IPayloadBuilder(builder);

    // verify the withdrawal can be achieved
    verifyWithdrawal(params, payloadBuilder, signature);

    // [...]
}
```

There is no security impact of this issue. Calling with an invalid chainId or depositary will simply cause a reversion. But it is still best practice to handle it properly so that errors are more clear.

4.2. TON escrow coding practices

The TON escrow contract can be optimized in a number of ways.

First, there is some inconsistency in how state is saved. The deposit operation opts to call `save_data`, despite not mutating state at all.

```
if (op == op::deposit) { ;; custom op for TON deposit with ID
    int deposit_id = in_msg_body~load_uint(64);
    cell event_data = begin_cell()
```

```

        .store_uint(event::deposit, 32)
        .store_uint(0, 1)           ;; Asset type
        .store_slice(my_address())  ;; Wallet
        .store_coins(msg_value)     ;; Amount
        .store_slice(sender)        ;; Depositor address
        .store_uint(deposit_id, 64) ;; ID
        .end_cell();

    send_log_message(event_data.begin_parse());
    save_data();
    return ();
}

```

The transfer notification also does not touch state and does not include a call to `save_data`.

```

if (op == op::transfer_notification) {
    int amount = in_msg_body~load_coins(); ;; amount
    slice from_address = in_msg_body~load_msg_addr(); ;; sender address
    int deposit_id = 0;
    if (in_msg_body.slice_refs_empty?() == 0) {
        cell payload = in_msg_body~load_ref();
        slice ps = payload.begin_parse();
        if (ps.slice_bits() >= 64) {
            deposit_id = ps~load_uint(64);
        }
    }

    cell event_data = begin_cell()
        .store_uint(event::deposit, 32)
        .store_uint(1, 1)           ;; Asset type
        .store_slice(sender)        ;; Jetton Wallet
        .store_coins(amount)        ;; Amount
        .store_slice(from_address)  ;; Depositor address
        .store_uint(deposit_id, 64) ;; Id
        .end_cell();

    send_log_message(event_data.begin_parse());
    return ();
}

```

The contract may be easier to understand if `save_data` were used consistently.

Additionally, the nonce is checked two separate times in `process_transfer`.

```

throw_if(error::invalid_nonce, msg_nonce <= nonce);

```

```
throw_unless(error::invalid_nonce, msg_nonce == nonce + 1);
```

The first check is nearly always dominated by the second. We recommend deleting the first check to simplify the code and reduce gas usage, as the first check is unnecessary.

4.3. TON escrow execution has loose ordering

By enforcing that the nonce increases by exactly one during each transfer, the escrow contract appears to require approved actions to be performed in a strict order. The nonce is updated as follows.

```
throw_if(error::invalid_nonce, msg_nonce <= nonce);  
throw_unless(error::invalid_nonce, msg_nonce == nonce + 1);  
  
nonce = msg_nonce;
```

This might not actually be the case, because messages to different contracts have no guarantee of being processed in order. This limitation makes it similar to the EVM implementation, which does not provide any mechanism for enforcing the transaction order.

4.4. TON escrow implementation has fewer features

We wanted to note two limitations of the TON implementation.

The Solidity implementation of the depository contract has more expressive configuration compared to the TON version. For instance, ownership can be changed in the EVM version, but the TON contract fixes the owner at deployment.

Additionally, EVM gives the allocator more flexibility in verifying signatures, specifically by checking validity in an external call. But the TON contract bakes signature verification into the escrow contract.

5. Threat Model

This provides a full threat model description for various functions. As time permitted, we analyzed each function in the files and created a written threat model for some critical functions. A threat model documents a given function's externally controllable inputs and how an attacker could leverage each input to cause harm.

Not all functions in the audit scope may have been modeled. The absence of a threat model in this section does not necessarily suggest that a function is safe.

5.1. Module: RelayAllocator.sol

Function: `grantRole(byte[32] role, address newOwner)`

This function grants a role to an address.

Inputs

- `role`
 - **Control:** Controllable by the owner.
 - **Constraints:** None.
 - **Impact:** The role granted to the address.
- `newOwner`
 - **Control:** Controllable by the owner.
 - **Constraints:** None.
 - **Impact:** The address granted the role.

Branches and code coverage

Intended branches

- Grant the role to the address.
- ☒ Test coverage

Negative behavior

- Revert if the caller is not the owner.
- ☒ Negative test

Function: `init()`

This function initializes the XCC subaccount for the contract.

Function: `revokeRole(byte[32] role, address newOwner)`

This function revokes a role from an address.

Inputs

- `role`
 - **Control:** Controllable by the owner.
 - **Constraints:** None.
 - **Impact:** The role revoked from the address.
- `newOwner`
 - **Control:** Controllable by the owner.
 - **Constraints:** None.
 - **Impact:** The address revoked from the role.

Branches and code coverage**Intended branches**

- Revoke the role from the address.

☒ Test coverage

Negative behavior

- Revert if the caller is not the owner.

☒ Negative test

Function: `setDelay(uint256 _delay)`

This function sets the global delay for withdrawal requests.

Inputs

- `_delay`
 - **Control:** Controllable by the owner.
 - **Constraints:** None.
 - **Impact:** The global delay for withdrawal requests.

Branches and code coverage

Intended branches

- Set the global delay for withdrawal requests.
- ☒ Test coverage

Negative behavior

- Revert if the caller is not the owner.
- ☒ Negative test

Function: `setDepositoryDelay(uint256 chainId, string depository, uint256 _delay)`

This function sets the delay for a specific chain and depository.

Inputs

- `chainId`
 - **Control:** Controllable by the owner.
 - **Constraints:** None.
 - **Impact:** The chain ID where the delay is applied.
- `depository`
 - **Control:** Controllable by the owner.
 - **Constraints:** None.
 - **Impact:** The depository where the delay is applied.
- `_delay`
 - **Control:** Controllable by the owner.
 - **Constraints:** None.
 - **Impact:** The delay in seconds.

Branches and code coverage

Intended branches

- Set the delay for the specific chain and depository.
- ☒ Test coverage

Negative behavior

- Revert if the caller is not the owner.

☒ Negative test

Function: `setHub(address _hub)`

This function sets the hub contract address.

Inputs

- `_hub`
 - **Control:** Controllable by the owner.
 - **Constraints:** None.
 - **Impact:** The hub contract address.

Branches and code coverage

Intended branches

- Set the hub contract address.

☒ Test coverage

Negative behavior

- Revert if the caller is not the owner.

☒ Negative test

Function: `setPayloadBuilder(uint256 chainId, string depository, address builder)`

This function sets the payload builder for a specific chain and depository.

Inputs

- `chainId`
 - **Control:** Controllable by the owner.
 - **Constraints:** None.
 - **Impact:** The chain ID that the payload builder is set for.
- `depository`
 - **Control:** Controllable by the owner.

- **Constraints:** None.
 - **Impact:** The depository that the payload builder is set for.
- builder
 - **Control:** Controllable by the owner.
 - **Constraints:** None.
 - **Impact:** The payload builder contract address.

Branches and code coverage

Intended branches

- Set the payload builder for the given chain and depository.
- ☒ Test coverage

Negative behavior

- Revert if the caller is not the owner.
- ☒ Negative test

Function: `setSignatureFee(uint256 _fee)`

This function sets the signature fee in wNEAR.

Inputs

- `_fee`
 - **Control:** Controllable by the owner.
 - **Constraints:** None.
 - **Impact:** The signature fee in wNEAR.

Branches and code coverage

Intended branches

- Set the signature fee in wNEAR.
- ☒ Test coverage

Negative behavior

- Revert if the caller is not the owner.
- ☐ Negative test

Function: `signWithdrawCallback(byte[32] withdrawRequestHash, byte[32] hashToSign)`

This function is called by the NEAR signer account after the signing is complete.

Inputs

- `withdrawRequestHash`
 - **Control:** Controllable by the near signer account.
 - **Constraints:** None.
 - **Impact:** The hash of the requested withdrawal payload that was signed.
- `hashToSign`
 - **Control:** Controllable by the near signer account.
 - **Constraints:** None.
 - **Impact:** The hash to sign.

Branches and code coverage**Intended branches**

- Insert the signed payload into the `signedPayloads` mapping.
 - ☒ Test coverage

Negative behavior

- Revert if the caller is not the near signer account.
 - ☒ Negative test

Function: `signWithdrawPayloadHash(SubmitWithdrawRequest params, bytes signature, GasSettings gasSettings, uint32 hashIndex)`

This function triggers the signing of a previously submitted withdraw request.

Inputs

- `params`
 - **Control:** Full control.
 - **Constraints:** None.
 - **Impact:** The withdraw-request parameters.
- `signature`

- **Control:** Full control.
 - **Constraints:** None.
 - **Impact:** The signature of the withdraw request.
- gasSettings
 - **Control:** Full control.
 - **Constraints:** None.
 - **Impact:** The gas settings for the withdraw request.
- hashIndex
 - **Control:** Full control.
 - **Constraints:** None.
 - **Impact:** The index of the hash to sign for this request.

Branches and code coverage

Intended branches

- Transfer the signature fee to the contract if it is not zero.
 - ☒ Test coverage
- Verify the withdrawal can be achieved.
 - ☒ Test coverage
- Call the RelayHub contract to transfer the tokens from the spender to the contract.
 - ☒ Test coverage
- Call the PayloadBuilder contract to get the hash to sign.
 - ☒ Test coverage
- Call the ChainSignatures contract to sign the hash.
 - ☒ Test coverage
- Call the transact function to sign the hash.
 - ☒ Test coverage

Negative behavior

- Revert if the payload does not exist.
 - ☒ Negative test
- Revert if the payload is not ready to be signed.
 - ☒ Negative test
- Revert if the caller is not approved to withdraw.
 - ☒ Negative test

- Revert if the payload is already signed.
- Negative test

Function call analysis

- `this.near.wNEAR.transferFrom(msg.sender, address(this), this.signatureFee)`
 - **What is controllable?** `msg.sender`.
 - **If the return value is controllable, how is it used and how can it go wrong?** The return value is discarded.
 - **What happens if it reverts, reenters or does other unusual control flow?** No problem.
- `this.verifyWithdrawal(params, payloadBuilder, signature) -> payloadBuilder.family()`
 - **What is controllable?** `payloadBuilder` is partially controllable.
 - **If the return value is controllable, how is it used and how can it go wrong?** `tokenId` and `spenderAlias` are partially controllable.
 - **What happens if it reverts, reenters or does other unusual control flow?** No problem.
- `this.verifyWithdrawal(params, payloadBuilder, signature) -> Utils.generateTokenId(family, params.chainId, params.currency)`
 - **What is controllable?** `family` is partially controllable; `params.chainId` and `params.currency` are controllable.
 - **If the return value is controllable, how is it used and how can it go wrong?** `tokenId` is partially controllable.
 - **What happens if it reverts, reenters or does other unusual control flow?** No problem.
- `this.verifyWithdrawal(params, payloadBuilder, signature) -> Utils.generateAddress(family, params.chainId, params.receiver)`
 - **What is controllable?** `family` is partially controllable; `params.chainId` and `params.receiver` are controllable.
 - **If the return value is controllable, how is it used and how can it go wrong?** `spenderAlias` is partially controllable.
 - **What happens if it reverts, reenters or does other unusual control flow?** No problem.
- `this.verifyWithdrawal(params, payloadBuilder, signature) -> RelayHub(this.hub).isOperator(params.spender, msg.sender)`
 - **What is controllable?** `params.spender` and `msg.sender`.
 - **If the return value is controllable, how is it used and how can it go wrong?** Approval check can go wrong and allow a nonapproved caller to withdraw.

- **What happens if it reverts, reenters or does other unusual control flow?** No problem.
- `this.verifyWithdrawal(params, payloadBuilder, signature) -> this.signatureMatchesReceiver(params, signature) -> Utils.toAddress(params.receiver)`
 - **What is controllable?** `params.receiver` is controllable.
 - **If the return value is controllable, how is it used and how can it go wrong?** The return value is not fully controllable.
 - **What happens if it reverts, reenters or does other unusual control flow?** No problem.
- `this.verifyWithdrawal(params, payloadBuilder, signature) -> this.signatureMatchesReceiver(params, signature) -> this._hashTypedDataV4(keccak256(abi.encode(RelayAllocator.PAYLOAD_TYPEHASH, params.chainId, keccak256(bytes(params.depositary)), keccak256(bytes(params.currency)), params.amount, params.spender, keccak256(bytes(params.receiver)), keccak256(params.data), params.nonce))) -> MessageHashUtils.toTypedDataHash(this._domainSeparatorV4(), structHash)`
 - **What is controllable?** `params`.
 - **If the return value is controllable, how is it used and how can it go wrong?** The return value is not fully controllable because of the `keccak256` calls.
 - **What happens if it reverts, reenters or does other unusual control flow?** No problem.
- `this.verifyWithdrawal(params, payloadBuilder, signature) -> this.signatureMatchesReceiver(params, signature) -> ECDSA.recover(digest, signature)`
 - **What is controllable?** None.
 - **If the return value is controllable, how is it used and how can it go wrong?** Not controllable.
 - **What happens if it reverts, reenters or does other unusual control flow?** No problem.
- `this.verifyWithdrawal(params, payloadBuilder, signature) -> RelayHub(this.hub).transferFrom(params.spender, address(this), tokenId, params.amount)`
 - **What is controllable?** `params`.
 - **If the return value is controllable, how is it used and how can it go wrong?** The return value is discarded.
 - **What happens if it reverts, reenters or does other unusual control flow?** No problem.
- `payloadBuilder.hashToSign(params.chainId, params.depositary, payload, hashIndex)`

- **What is controllable?** params and hashIndex are controllable, but payload is partially controllable.
 - **If the return value is controllable, how is it used and how can it go wrong?** The return value is not fully controllable because of the keccak256 calls.
 - **What happens if it reverts, reenters or does other unusual control flow?** No problem.
- this._signUsingChainSignatures(withdrawRequestHash, hashToSign, payloadBuilder, gasSettings) -> ChainSignatures.encodeJSONRequest(hashToSign, payloadBuilder.curve(), this.signerPath, keccak256(abi.encodePacked(payloadBuilder.curve()))) == RelayAllocator.ECDSA_HASH ? "0" : "1")
 - **What is controllable?** hashToSign and payloadBuilder are partially controllable.
 - **If the return value is controllable, how is it used and how can it go wrong?** The message to sign can be tampered with but is not fully controllable.
 - **What happens if it reverts, reenters or does other unusual control flow?** No problem.
- this._signUsingChainSignatures(withdrawRequestHash, hashToSign, payloadBuilder, gasSettings) -> payloadBuilder.curve()
 - **What is controllable?** payloadBuilder is partially controllable.
 - **If the return value is controllable, how is it used and how can it go wrong?** The return value is not fully controllable.
 - **What happens if it reverts, reenters or does other unusual control flow?** No problem.
- this._signUsingChainSignatures(withdrawRequestHash, hashToSign, payloadBuilder, gasSettings) -> payloadBuilder.curve()
 - **What is controllable?** payloadBuilder is partially controllable.
 - **If the return value is controllable, how is it used and how is it used?** The return value is not fully controllable.
 - **What happens if it reverts, reenters or does other unusual control flow?** No problem.
- this._signUsingChainSignatures(withdrawRequestHash, hashToSign, payloadBuilder, gasSettings) -> AuroraSdk.call(this.near, this.nearSigner, "sign", data, 1, gasSettings.signGas)
 - **What is controllable?** data and gasSettings.signGas.
 - **If the return value is controllable, how is it used and how is it used?** The return value is not fully controllable.
 - **What happens if it reverts, reenters or does other unusual control flow?** No problem.
- this._signUsingChainSignatures(withdrawRequestHash, hashToSign, payloadBuilder, gasSettings) -> AuroraSdk.auroraCall(this.near, address(this),

```
abi.encodeWithSelector(RelayAllocator.signWithdrawCallback.selector,
withdrawRequestHash, hashToSign), 0, gasSettings.callbackGas)
```

- **What is controllable?** withdrawRequestHash and hashToSign are partially controllable; gasSettings.callbackGas is fully controllable.
- **If the return value is controllable, how is it used and how can it go wrong?** The return value is not fully controllable.
- **What happens if it reverts, reenters or does other unusual control flow?** No problem.
- `this._signUsingChainSignatures(withdrawRequestHash, hashToSign, payloadBuilder, gasSettings) -> AuroraSdk.transact(AuroraSdk.then(callSign, callback))`
 - **What is controllable?** callSign and callback are partially controllable.
 - **If the return value is controllable, how is it used and how can it go wrong?** The return value is discarded.
 - **What happens if it reverts, reenters or does other unusual control flow?** No problem.

Function: submitAndSignWithdrawRequest(SubmitWithdrawRequest params, bytes signature, GasSettings gasSettings)

This function submits a withdraw request to the payload builder, stores the returned payload, and triggers its signature immediately.

Inputs

- params
 - **Control:** Full control.
 - **Constraints:** None.
 - **Impact:** The withdraw-request parameters.
- signature
 - **Control:** Full control.
 - **Constraints:** None.
 - **Impact:** The signature of the withdraw request.
- gasSettings
 - **Control:** Full control.
 - **Constraints:** None.
 - **Impact:** The gas settings for the withdraw request.

Branches and code coverage

Intended branches

- Call the `signWithdrawPayloadHash` function.

☒ Test coverage

Negative behavior

- Revert if the delay is set.

☒ Negative test

Function: `submitWithdrawRequest` (`SubmitWithdrawRequest` params)

This function is a wrapper around the `_submitWithdrawRequest` function that returns the hash of the withdrawal request.

Inputs

- `params`
 - **Control:** Full control.
 - **Constraints:** None.
 - **Impact:** The withdraw-request parameters.

Branches and code coverage

Intended branches

- Call the `_submitWithdrawRequest` function.

☒ Test coverage

Function: `suspend` (`address` `withdrawer`)

This function prevents a withdrawer from withdrawing.

Inputs

- `withdrawer`
 - **Control:** Controllable by the multi-sig owner.
 - **Constraints:** None.

- **Impact:** The withdrawer prevented from withdrawing.

Branches and code coverage

Intended branches

- Revoke the role from the address.

☒ Test coverage

Negative behavior

- Revert if the caller is not the multi-sig owner.

☒ Negative test

Function: `withdrawToNear(uint256 amount)`

This function withdraws the wNEAR balance of this contract from the Aurora contract to the NEAR network.

Inputs

- `amount`
 - **Control:** Full control.
 - **Constraints:** None.
 - **Impact:** The amount of wNEAR to withdraw.

Branches and code coverage

Intended branches

- Call the IEvmERC20 contract to withdraw the wNEAR balance from the Aurora contract to the NEAR network.

☒ Test coverage

- Call the `near.call` contract to unwrap the wNEAR on the NEAR network.

☒ Test coverage

- Call the `transact` function to unwrap the wNEAR on the NEAR network.

☒ Test coverage

Function: `_submitWithdrawRequest(SubmitWithdrawRequest params)`

This function submits a withdraw request to the payload builder, stores the returned payload, and returns the hash of the withdrawal request.

Inputs

- `params`
 - **Control:** Full control.
 - **Constraints:** None.
 - **Impact:** The withdraw-request parameters.

Branches and code coverage**Intended branches**

- Build the payload.
 - ☒ Test coverage
- Store the hash of the request for deduplication and verification.
 - ☒ Test coverage
- Use the depository-specific delay if set; otherwise, fall back to global delay.
 - ☒ Test coverage
- Set the payload timestamp.
 - ☒ Test coverage
- Emit the `PayloadBuilt` event.
 - ☒ Test coverage

Negative behavior

- Revert if the payload builder is not set.
 - ☒ Negative test

5.2. Module: `RelayDepository.sol`**Function: `depositErc20(address depositor, address token, uint256 amount, byte[32] id)`**

This function allows the depositor to deposit ERC-20 tokens.

Inputs

- `depositor`
 - **Control:** Full control.
 - **Constraints:** None.
 - **Impact:** The address will be credited with the ERC-20 tokens.
- `token`
 - **Control:** Full control.
 - **Constraints:** None.
 - **Impact:** The address of the ERC-20 token that is being deposited.
- `amount`
 - **Control:** Full control.
 - **Constraints:** None.
 - **Impact:** The amount of tokens being deposited.
- `id`
 - **Control:** Full control.
 - **Constraints:** None.
 - **Impact:** The identifier associated with the deposit.

Branches and code coverage

Intended branches

- Transfer the tokens to the contract.
 - ☒ Test coverage
- Emit the `RelayErc20Deposit` event.
 - ☒ Test coverage

Negative behavior

- Revert if the token does not exist.
 - ☐ Negative test

Function: `depositErc20(address depositor, address token, byte[32] id)`

This function is a wrapper around the `depositErc20` function that uses the full allowance granted to this contract.

Inputs

- depositor
 - **Control:** Full control.
 - **Constraints:** None.
 - **Impact:** The address that will be credited with the ERC-20 tokens.
- token
 - **Control:** Full control.
 - **Constraints:** None.
 - **Impact:** The address of the ERC-20 token that is being deposited.
- id
 - **Control:** Full control.
 - **Constraints:** None.
 - **Impact:** The identifier associated with the deposit.

Branches and code coverage

Intended branches

- Call the `depositErc20` function with the full allowance.
- ☒ Test coverage

Function: `depositNative(address depositor, byte[32] id)`

This function allows the depositor to deposit native tokens.

Inputs

- depositor
 - **Control:** Full control.
 - **Constraints:** None.
 - **Impact:** The address that will be credited with the native tokens.
- id
 - **Control:** Full control.
 - **Constraints:** None.
 - **Impact:** The identifier associated with the deposit.

Branches and code coverage

Intended branches

- Emit the `RelayNativeDeposit` event.
- ☒ Test coverage

Function: `execute(CallRequest request, bytes signature)`

This function allows the allocator to execute a `CallRequest` signed by the allocator.

Inputs

- `request`
 - **Control:** Full control.
 - **Constraints:** The expiration must be in the future.
 - **Impact:** The `CallRequest` that is being executed.
- `signature`
 - **Control:** Full control.
 - **Constraints:** The signature must be valid.
 - **Impact:** The signature that is used to verify the `CallRequest`.

Branches and code coverage

Intended branches

- Update the call request's mapping.
- ☒ Test coverage
- Execute the calls.
- ☒ Test coverage
- Emit the `RelayCallExecuted` event if the call is successful.
- ☒ Test coverage

Negative behavior

- Revert if the call-request expiration is in the past.
- ☐ Negative test
- Revert if the allocator signature is invalid.
- ☒ Negative test

- Revert if the call request has already been used.
 - ☐ Negative test
- Revert if the call fails and failure is not allowed.
 - ☐ Negative test

Function call analysis

- `this._hashCallRequest(request) -> this._hashTypedData(structHash)`
 - **What is controllable?** `request`.
 - **If the return value is controllable, how is it used and how can it go wrong?**
An attacker will be able to execute arbitrary code.
 - **What happens if it reverts, reenters or does other unusual control flow?** No problem.
- `SignatureCheckerLib.isValidSignatureNow(this._allocator, eip712Hash, signature)`
 - **What is controllable?** `eip712Hash` and `signature`.
 - **If the return value is controllable, how is it used and how can it go wrong?**
An attacker will be able to execute arbitrary code, but the signature must be valid.
 - **What happens if it reverts, reenters or does other unusual control flow?** No problem.

Function: `setAllocator(address _allocator)`

This function allows the owner to set the allocator address.

Inputs

- `_allocator`
 - **Control:** Controllable by the owner.
 - **Constraints:** The allocator address must be nonzero.
 - **Impact:** The allocator address that is used to sign the call requests.

Branches and code coverage

Intended branches

- Update the allocator address.
 - ☒ Test coverage

Negative behavior

- Revert if the caller is not the owner.
 - ☒ Negative test
- Revert if the allocator address is zero.
 - ☐ Negative test

5.3. Module: RelayMultisigSigner.sol

Function: `approveSignature(byte[32] hashToSign, string curve)`

This function approves a signature for a given hash and curve.

Inputs

- `hashToSign`
 - **Control:** Full control.
 - **Constraints:** None.
 - **Impact:** The hash of the message to sign.
- `curve`
 - **Control:** Full control.
 - **Constraints:** None.
 - **Impact:** The curve to use for signing.

Branches and code coverage

Intended branches

- Set the approved signature for the given hash and curve.
 - ☐ Test coverage

Negative behavior

- Revert if the caller is not the owner.
 - ☐ Negative test

Function: `init()`

This function initializes the XCC subaccount for the contract.

Function: `signatureCallback(string curve, byte[32] hashToSign)`

This function is called by the NEAR runtime when the signature is ready.

Inputs

- `curve`
 - **Control:** Full control.
 - **Constraints:** None.
 - **Impact:** The curve used for signing.
- `hashToSign`
 - **Control:** Full control.
 - **Constraints:** None.
 - **Impact:** The hash of the message that was signed.

Branches and code coverage

Intended branches

- Update the signatures mapping with the signature.
- ☐ Test coverage

Negative behavior

- Revert if the caller is not the NEAR runtime.
- ☐ Negative test
- Revert if the promise result is not successful.
- ☐ Negative test

Function call analysis

- `AuroraSdk.nearRepresentativeImplicitAddress(address(this))`
 - **What is controllable?** Not controllable.
 - **If the return value is controllable, how is it used and how can it go wrong?** Not controllable.
 - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `AuroraSdk.promiseResult(0)`
 - **What is controllable?** Not controllable.

- **If the return value is controllable, how is it used and how can it go wrong?**
Not controllable.
- **What happens if it reverts, reenters or does other unusual control flow?**
N/A.

Function: `sign(byte[32] hashToSign, string curve, uint64 signGas, uint64 callbackGas)`

This function signs a given hash and curve.

Inputs

- `hashToSign`
 - **Control:** Full control.
 - **Constraints:** The hash must be in the approved list.
 - **Impact:** The hash of the message to sign.
- `curve`
 - **Control:** Full control.
 - **Constraints:** The curve must be in the approved list.
 - **Impact:** The curve to use for signing.
- `signGas`
 - **Control:** Full control.
 - **Constraints:** None.
 - **Impact:** The amount of NEAR gas to use for the signing request.
- `callbackGas`
 - **Control:** Full control.
 - **Constraints:** None.
 - **Impact:** The amount of NEAR gas to use for the callback.

Branches and code coverage

Intended branches

- Call the XCC precompile to sign the payload via the `transact` function.

☒ Test coverage

Negative behavior

- Revert if the hash to sign is not in the approved list.

☑ Negative test

Function call analysis

- ChainSignatures.encodeJSONRequest(hashToSign, curve, this.signerPath, domainId)
 - **What is controllable?** hashToSign and curve are fully controllable, but signerPath and domainId are not.
 - **If the return value is controllable, how is it used and how can it go wrong?** It is partially controllable.
 - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- AuroraSdk.call(this.near, this.nearSigner, "sign", signatureRequest, 1, signGas)
 - **What is controllable?** signatureRequest is partially controllable, and signGas is fully controllable.
 - **If the return value is controllable, how is it used and how can it go wrong?** High signGas would revert.
 - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- AuroraSdk.auroraCall(this.near, address(this), abi.encodeWithSelector(RelayMultisigSigner.signatureCallback.selector, curve, hashToSign), 0, callbackGas)
 - **What is controllable?** curve and hashToSign are fully controllable, and callbackGas is fully controllable.
 - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
 - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- AuroraSdk.transact(AuroraSdk.then(callSign, callback))
 - **What is controllable?** callSign and callback are partially controllable.
 - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
 - **What happens if it reverts, reenters or does other unusual control flow?** N/A.

6. Assessment Results

During our assessment on the scoped Relay Settlement Protocol files, we discovered 11 findings. One critical issue was found. Two were of high impact, five were of medium impact, two were of low impact, and the remaining finding was informational in nature.

While the overall code quality is acceptable, several concerns lead us to conclude that the system is not yet ready for production.

First, the volume of issues identified during the assessment prevents us from having full confidence in the system's readiness. Put simply, the number of findings suggests there are likely additional bugs we have not uncovered yet. Remediating these vulnerabilities will require modifications across the codebase. Given the current complexity and statefulness of the system, it is difficult to guarantee that additional issues will not be introduced during that process.

Additionally, as discussed in detail in finding [3.9.7](#) and section [5.7](#), the existing test suite is, in our view, insufficient. Please refer to the finding for details regarding this recommendation.

After addressing the reported vulnerabilities and strengthening the test suite, we recommend undergoing another comprehensive assessment — either with our team or another reputable security firm. This follow up review should include the scope we assessed as well as all related contracts and supporting systems (e.g. the RelayHub and off-chain, automated software such as the oracle).

6.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.